Industrial Practice
Topic: Online Banking System

Roll Number	Participants	Signature
44	Hiral P Patel	
45	Ayush V Patil	
46	Ayush A Patil	

Under the

Guidance of:

Mr. Jignesh Patel

Department of

Artificial Intelligence & Data Science

Abstract:-

The Online Banking System using Spring Boot is a secure and user-friendly web application designed to simulate the core functionalities of a modern banking system. The system enables users to create accounts, deposit and withdraw funds, transfer money between accounts, and view transaction summaries. The project leverages Spring Boot for backend development, MySQL for database management, and a simple web interface built using HTML, CSS, and JavaScript. The primary objective is to understand real-world banking processes while implementing a scalable and maintainable architecture suitable for enterprise-level systems.

Introduction:-

In today's digital era, online banking systems play a critical role in enabling customers to perform financial transactions conveniently and securely. This project was developed as part of an internship to gain practical experience in developing enterprise web applications using Spring Boot. The Online Banking System provides fundamental operations such as account management, deposits, withdrawals, and fund transfers.
The system emphasizes security, modular design, and data integrity, which are essential aspects of financial applications.

Project Description

The Online Banking System was developed using the Spring Boot framework due to its efficiency in building Java-based web applications with embedded server support. The backend is responsible for managing the business logic, while the frontend provides a clean and responsive interface for user interaction. The MySQL database stores account and transaction details.

System Architecture

The architecture follows the Model-View-Controller (MVC) pattern:
• Model: Represents the business logic and data persistence layer using Spring Data JPA.
• View: Implemented using HTML, CSS, and JavaScript for the user interface.
• Controller: Handles HTTP requests and interacts with the model to update or fetch data.

Modules

1. **User Authentication Module –** Handles user login and session management.
2. **Account Management Module**– Allows users to create new accounts and view account details.
3. **Transaction Module**– Enables deposit, withdrawal, and fund transfer between accounts.
4. **Admin Module** – Provides administrative access to manage users and monitor transactions.
5. **Report Module** – Displays transaction history and summary reports.

Database Design

The database consists of the following main tables:
• Users (user_id, username, password, email)
• Accounts (account_no, type, balance, user_id)
• Transactions (transaction_id, account_no, type, amount, date)

Results and Discussion

The application was successfully deployed and tested locally using Spring Boot's embedded Tomcat server.
Users were able to log in, perform deposits and withdrawals, and transfer funds between accounts.
The interface provides real-time balance updates and transaction summaries.

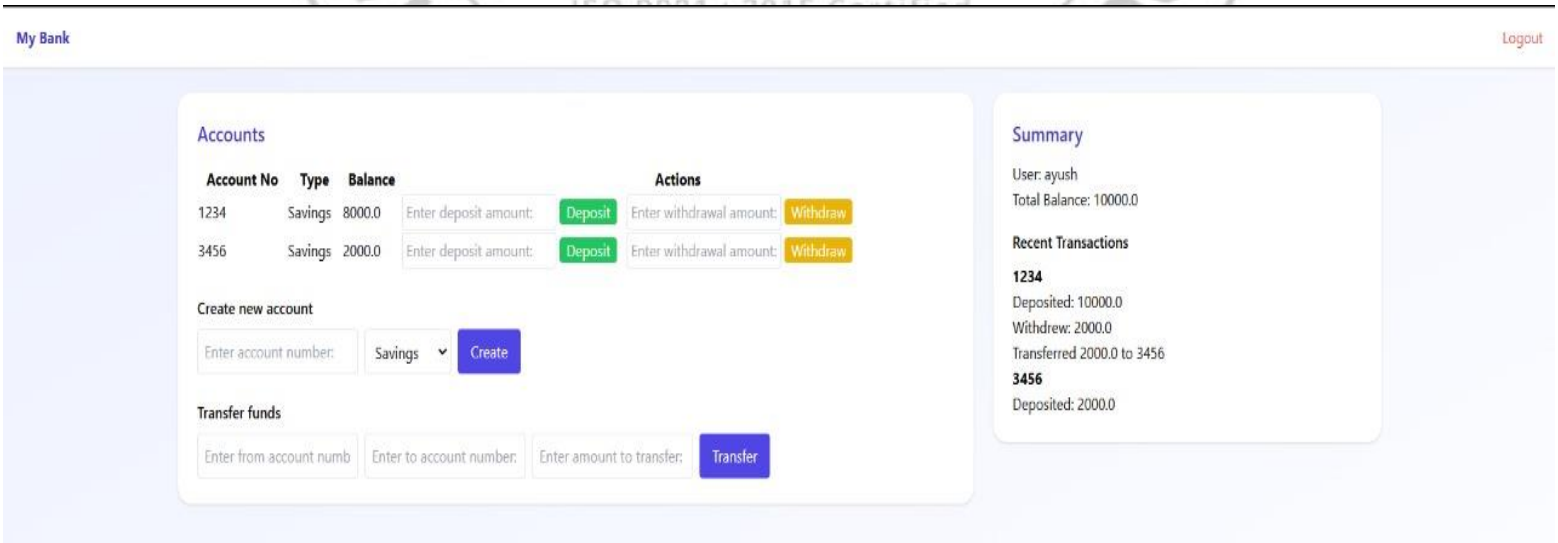

Figure 1: User dashboard showing account summary and recent transactions.

Conclusion and Future Scope:-

The Online Banking System successfully demonstrates how modern banking operations can be implemented using Spring Boot and MySQL.
It provides a robust foundation for understanding real-world banking application development.
The project enhanced practical knowledge of Java frameworks, MVC design, RESTful APIs, and database handling.

Future Enhancements:-

- Integration of email/SMS notifications for transactions.
- Implementing security features like OTP and encryption.
- Mobile application version using React Native or Flutter.
- Incorporating AI-based fraud detection and analytics.

References

References (Updated with Links)

1. **Spring Boot Documentation**
   https://spring.io/projects/spring-boot
   → Official Spring Boot guide used for application setup, REST API creation, and dependency management.
2. **Spring Data JPA Documentation**
   https://spring.io/projects/spring-data-jpa
   → Used for integrating the database layer and simplifying CRUD operations.
3. **MySQL Documentation**
   https://dev.mysql.com/doc/
   → Reference for creating database schemas, writing SQL queries, and managing relational data.
4. **Thymeleaf Official Documentation**
   https://www.thymeleaf.org/documentation.html
   → Used for implementing server-side rendering of dynamic web pages in Spring Boot.
5. **Spring Security Guide**
   https://spring.io/guides/topicals/spring-security-architecture
   → Reference for implementing secure authentication and authorization.
6. **Bootstrap Official Documentation**
   https://getbootstrap.com/docs/
   → For styling the front-end and creating responsive UI components.

7. **Java SE Documentation**
   https://docs.oracle.com/javase/
   → Used to reference Java core libraries, exception handling, and OOP features.
8. **Baeldung – Spring Boot Tutorials**
   https://www.baeldung.com/
   → For practical examples of Spring Boot CRUD, REST controllers, and application configuration.
9. **GeeksforGeeks – Spring Boot CRUD Example**
   https://www.geeksforgeeks.org/spring-boot-crud-operations/
   → Used for initial guidance in setting up controllers and repository logic.
10. **YouTube Tutorial – Spring Boot CRUD Application**
    https://www.youtube.com/watch?v=vtPkZShrvXQ
    → For visual reference on creating a full-stack Spring Boot + MySQL project.

Annexure

Backend:-

- **OnlineBankingApplication.java**: This is the entry point for your Spring Boot application.
- **MainController.java**: Handles HTTP requests, user actions, and routes between frontend and backend.
- **Phase2Phase3Service.java**: Contains business logic for banking operations (deposit, withdraw, transfer, transaction history).
- **Model files** (User.java, Account.java): Define the data structures used by the backend.

OnlineBankingSpringApp_Final > src > main > java > com > onlinebanking > OnlineBankingApplication.java > Language Support for J

```java
package com.onlinebanking;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class OnlineBankingApplication {
    Run | Debug
    public static void main(String[] args) {
        SpringApplication.run(OnlineBankingApplication.class, args);
    }
}
```

MainController.java 1

OnlineBankingSpringApp_Final > src > main > java > com > onlinebanking > controller > MainController.java > Language Support for Java(TM) by Re

```java
package com.onlinebanking.controller;

import com.onlinebanking.model.User;
import com.onlinebanking.model.Account;
import com.onlinebanking.service.Phase2Phase3Service;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import jakarta.servlet.http.HttpSession;

@Controller
public class MainController {
    private final Phase2Phase3Service service;

    public MainController(Phase2Phase3Service service) { this.service = service; }

    @GetMapping({"/", "/login"})
    public String login() { return "login"; }

    @PostMapping("/login")
    public String doLogin(@RequestParam String username, @RequestParam String password, HttpSession session
        User u = service.authenticate(username, password);
        if (u == null) { model.addAttribute("error", "Invalid credentials"); return "login"; }
        session.setAttribute("user", u.getUsername());
        return "redirect:/dashboard";
    }

    @GetMapping("/register")
    public String register() { return "register"; }

    @PostMapping("/register")
    public String doRegister(@RequestParam String username, @RequestParam String password, Model model){
        boolean ok = service.register(username, password);
        if (!ok) { model.addAttribute("error", "Username already exists"); return "register"; }
        model.addAttribute("msg", "Registration successful. Please login."); return "login";
    }
```

```java
    @GetMapping("/dashboard")
    public String dashboard(Model model, HttpSession session){
        String un = (String) session.getAttribute("user"); if (un == null) return "redirect:/login";
        User u = service.getUser(un); model.addAttribute("user", u); return "dashboard";
    }

    @PostMapping("/createAccount")
    public String createAccount(@RequestParam String accNum, @RequestParam String type, HttpSession session
        String un = (String) session.getAttribute("user");
        boolean ok = service.createAccount(un, accNum, type);
        if (!ok) model.addAttribute("error", "Account creation failed or account exists");
        return "redirect:/dashboard";
    }

    @PostMapping("/deposit")
    public String deposit(@RequestParam String accNum, @RequestParam double amount, HttpSession session, Mo
        boolean ok = service.deposit(accNum, amount);
        if (!ok) model.addAttribute("error", "Deposit failed");
        return "redirect:/dashboard";
    }

    @PostMapping("/withdraw")
    public String withdraw(@RequestParam String accNum, @RequestParam double amount, HttpSession session, M
        boolean ok = service.withdraw(accNum, amount);
        if (!ok) model.addAttribute("error", "Withdrawal failed: insufficient funds or invalid account");
        return "redirect:/dashboard";
    }

    @PostMapping("/transfer")
    public String transfer(@RequestParam String fromAcc, @RequestParam String toAcc, @RequestParam double a
        boolean ok = service.transfer(fromAcc, toAcc, amount);
        if (!ok) model.addAttribute("error", "Transfer failed: check accounts and balance");
        return "redirect:/dashboard";
    }

    @PostMapping("/logout")
```

Sample Code Snippet:-

```java
@PostMapping("/deposit/{accountNo}")
public String depositAmount(@PathVariable long accountNo, @RequestParam double amount) {
    Account account = accountRepository.findById(accountNo).orElseThrow();
    account.setBalance(account.getBalance() + amount);
    accountRepository.save(account);
    return "redirect:/dashboard";}
```

Estd 2001
ISO 9001 : 2015 Certified
NBA and NAAC Accredited